

Solution *Sketches* to Problem Set 1

When solving a problem you can use all the results seen in class or in the textbook, but nothing else. You are allowed to discuss problems with other students, but must write up your solutions on your own – do not read or copy the written solutions of others. You should try working on each problem on your own first. If you work with others on a problem, write down who your collaborators are at the beginning of your solution write-up.

Problem 1 (Practice with Sets, Functions, Notation) 10 points

1. (3 pts) Give an example of an element $s \in S$ from each of the following sets S :
 - (a) $S = \{0, 1\}^4$: **Answer (e.g.):** $(0, 0, 1, 1)$ or can write it as a string 0011
 - (b) $S = \{x, y, z\} \times \{1, 2, 3, 4\}$: **Answer (e.g.):** $(x, 3)$
 - (c) $S = \text{Powerset}(\{\alpha, \beta, \gamma\})$: **Answer (e.g.):** $\{\alpha, \beta\}$
2. (3 pts) If A is a set of size $|A| = 4$, and B is a set of size $|B| = 6$, what is the size of each of the following sets (you can write your answer as e.g., 2^{20} or 10^3 instead of an exact number):
 - (a) $A \times B$. **Answer:** 24
 - (b) $\text{Powerset}(A)$. **Answer:** $2^4 = 16$
 - (c) $\text{Powerset}(A \times B)$. **Answer:** 2^{24} .
3. (2 pt) How many possible different functions f of the form $f : \{0, 1\}^2 \rightarrow \{a, b, c, d\}$ are there? **Answer:** 4^4 . (For each of the $2^2 = 4$ inputs, there are 4 choices for the output)
4. (2 pts) Is the following statement true or false? Think about it carefully using the precise meaning of logical connectives and explain your answer:
“The following holds for all sets S : if the power-set of S contains exactly 6 elements, then S is the empty set.”

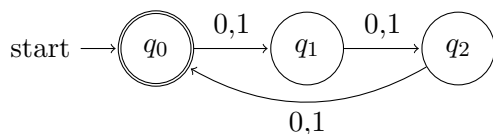
Answer: TRUE: The statement can be written as $\forall S : A(S) \Rightarrow B(S)$ where $A(S) = \text{“Powerset}(S) \text{ contains exactly 6 elements”}$ and $B(S) = \text{“}S \text{ is the empty set”}$. Now we notice that for any set S , the size of $\text{Powerset}(S)$ is always a power of 2 and therefore never 6. Therefore $A(S)$ is always FALSE and this in turn means that $A(S) \Rightarrow B(S)$ is always TRUE (doesn’t matter what $B(S)$ says at all). Therefore the entire statement is TRUE.

(Many students find this confusing and wrote that the statement is false. It’s useful to check that if the statement were false, this would mean that there exists some set S such that the power-set of S contains 6 elements and S is not the empty set. It’s clear that such a set cannot exist.)

Problem 2 (What Language is Recognized?)

10 Points

1. (5 pts) First, write out a formal description of this DFA using the notation $M = (Q, \Sigma, \delta, q_0, F)$ that we saw in class. Then describe what language does the DFA recognize.



Answer:

$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_{start} = q_0, F = \{q_0\})$$

$$\delta : \quad \delta(q_0, 0) = q_1 \quad \delta(q_0, 1) = q_1$$

$$\delta(q_1, 0) = q_2 \quad \delta(q_1, 1) = q_2$$

$$\delta(q_2, 0) = q_0 \quad \delta(q_2, 1) = q_0$$

The DFA above recognizes a binary language consisting of 0s and 1s, whose length must be a multiple of 3.

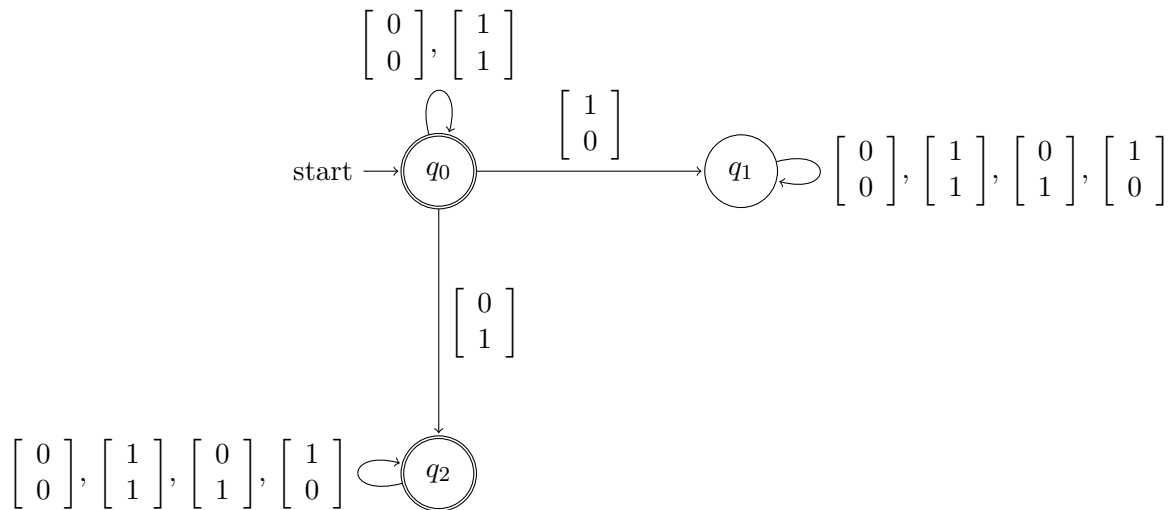
2. (5 pts) Let $\Sigma = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}$ be an alphabet.

When given a string $w \in \Sigma^*$ consider the top row as a number written in binary and consider the bottom row a second number written in binary.

So for example the string $w = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ represents the number 5 on top and 2 on the bottom.

Let $L = \{w \mid w \in \Sigma^* \text{ and the top binary number in } w \text{ is not greater than the bottom number in } w\}$. Construct a DFA for the language L .

Answer:



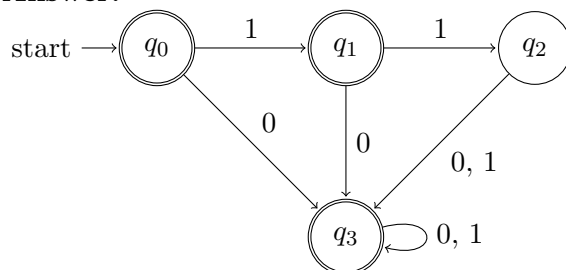
Problem 3 (Construct DFAs and NFAs)

20 Points

Fix the alphabet to be $\Sigma = \{0, 1\}$:

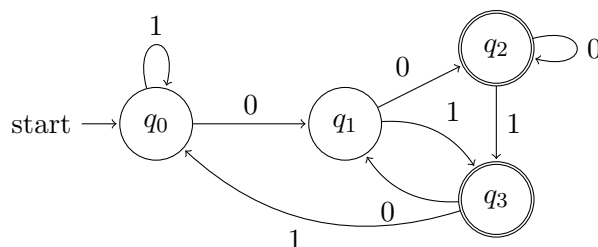
- (a) Construct a DFA that recognizes: $L = \{w | w \neq 11\}$.

Answer:



- (b) Construct a DFA that has 4 states and recognizes: $L = \{w | w \text{ ends with a } 01 \text{ or a } 00\}$.

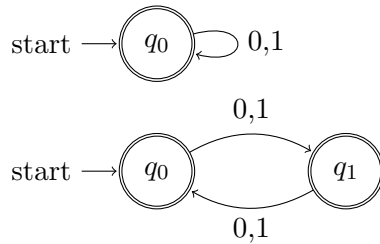
Answer:



- (c) Construct any two *different* DFAs M, M' that recognize the *same* language L . Try to make your example as simple as possible.

Answer:

$$L = \Sigma^*$$



Problem 4 (Finite languages are regular)

10 Points

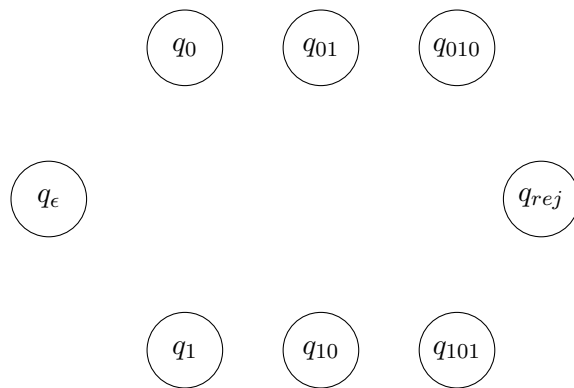
Show that if L is any language containing a finite number of strings (i.e., $|L|$ is finite) then L is regular. Do so by describing a general procedure that takes any such finite language L and shows how to construct a DFA that recognizes L (do not rely on NFAs or regular expressions). To illustrate that your general procedure works, give an example of what DFA it would produce if you started with the language $L = \{101, 10, 010\}$.

Answer: Let ℓ be the length of the longest string contained in L . Let's start with the case where the alphabet is $\Sigma = \{0, 1\}$. Then the DFA will look like a complete binary tree of height ℓ where every non-leaf state has 2 children and a transition labeled with 0 going to the left child and a transition labeled with 1 going to the right child. We can label the states of this DFA as q_w where $w \in \Sigma^*$ denotes the labels of the edges/transitions along the unique path from the root of the tree to the given node. So the root is denoted q_ϵ , its two children are q_0, q_1 , the two children of q_0 are q_{00}, q_{01} and the two children of q_1 are q_{10}, q_{11} , etc. After reading a string w , of length $|w| \leq \ell$, the DFA enters the state q_w . For every $w \in L$ we now make the state q_w an accepting state. In other words, the set of accepting states is $F = \{q_w : w \in L\}$. In the above solution, we can also “prune out” the subtrees which don't have any accepting states. This would make the DFA more compact. But since we just want to show existence, there is no need to do this.

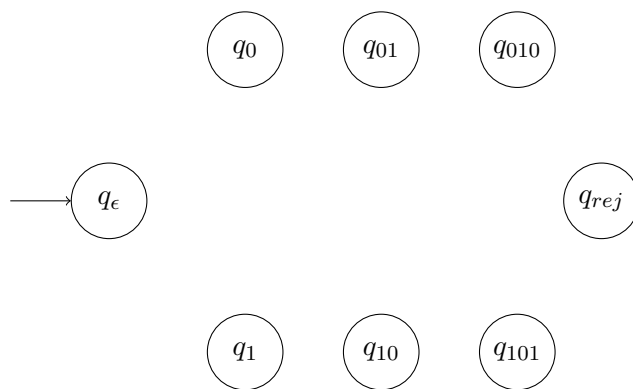
More generally, we can adapt the above to any alphabet Σ of size $|\Sigma| = p$ by making the DFA be a complete p -ary tree (each non-leaf node has p children) of height ℓ . Each non-leaf node has p children: each child corresponds to a different alphabet symbol and there is a transition going to that child with the corresponding alphabet symbol. As before, we can label the states of this DFA as q_w where $w \in \Sigma^*$ denotes the labels of the edges/transitions along the unique path from the root of the tree to the given node. We set the set of accepting states to be $F = \{q_w : w \in L\}$.

The above is not a complete DFA since the leaf states do not have transitions coming out of them. To fix this, we can add one more “sink state” which is not accepting and has all outgoing transitions looping back to itself. Then we add outgoing transitions going from the leaf states to the sink state.

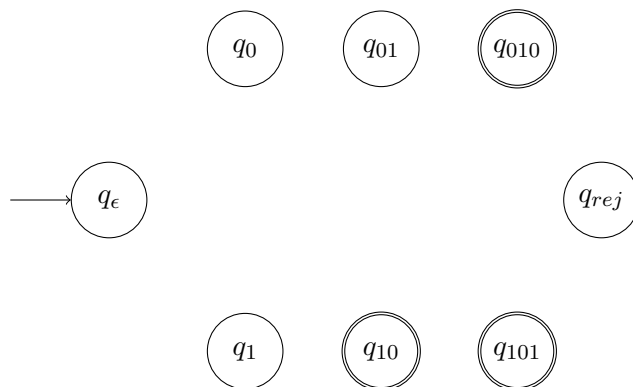
- ① The alphabet is the same as that in L .
- ② The states $Q = \{q_\epsilon, q_{rej}\} \cup \{q_s \mid s \text{ is prefix of a string in } L\}$.



③ The start state is q_ϵ .



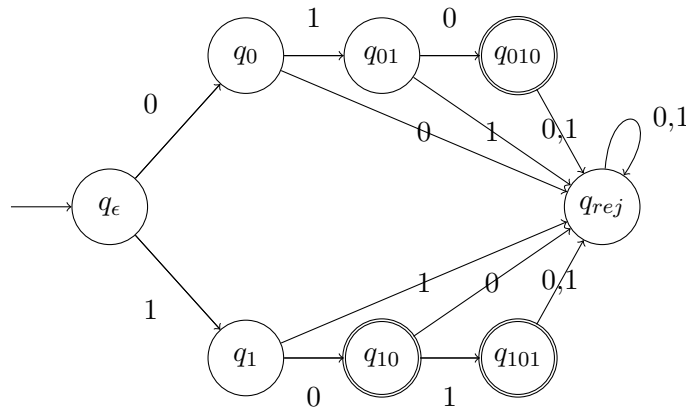
④ The accept states are $\{q_s \mid s \text{ is a string in } L\}$.



⑤ For transitions,

$$\delta(q_s, x) = \begin{cases} q_{sx}, & q_{sx} \in Q \\ q_{rej}, & \text{otherwise,} \end{cases}$$

$$\delta(q_{rej}, x) = q_{rej}.$$



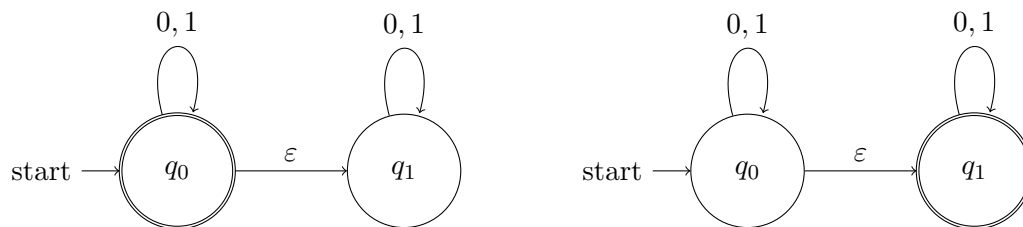
Problem 5 (Closure Properties)

10 Points

This question refers to results we showed in slide deck 2.

1. In class, we showed that if A is regular then its complement $\bar{A}$ is also regular. To do so, we took a DFA for A and converted it into a DFA for $\bar{A}$ by simply complementing the set of accept states. Would the same exact transformation also work for converting an NFA for A into an NFA for $\bar{A}$? Either prove that it works or give a counterexample.

Answer: The exact same transformation would **not** work. To see a counterexample, let N be the left NFA depicted below, which recognizes the language $L(N) = \{0, 1\}^*$. Then, applying the transformation from class (i.e., complementing the set of accept states) on N yields the NFA N' depicted below on the right. However, it is easy to see that $L(N') = \{0, 1\}^*$ which is not the complement of $L(N)$.



2. For the other closure properties, showing that if A, B are regular then so is $C = A \cup B$, $C = A \circ B$ and $C = A^*$, we started with DFAs for A, B and converted them into an NFA for C . For each of these transformations, would it also work if we started with NFAs for A, B instead of DFAs? (No need to prove your answer, just Yes/No for each one suffices.)

Answer: The answer is yes for all three transformations. (If you're following along in the Sipser book, the Theorems 1.45, 1.47, 1.49 there are already stated as converting NFAs for A, B into an NFA for C . In the slides from class, we stated these results as starting with DFAs, but we could have just as well started with NFAs.)